AEGIS: API Workflow and Data Delivery

Technical Deliverable

Ilias Akritidis*, Georgios Mallousis*, Vasileios Sevetlidis

International Hellenic University, Polytechnic School
Department of Computer, Informatics and Telecommunications Engineering

iliakrit@ihu.gr geomallo@ihu.gr vasiseve@ihu.gr

1 Purpose and scope

AEGIS uses ARGUS monitoring data to connect decisions made during gameplay with the preservation needs of five heritage sites. The API workflow supplies the site's risk scenarios, the instruments installed there, and recorded alarms that the game can explain. It also provides recent monitoring information for the opening screen. AEGIS implements these functions through a stable gameplay dataset and a separate mechanism for incorporating new records.

The scope encompasses snapshot acquisition, incremental collection, data preparation, publication and browser consumption.[1] Verification addresses the processing logic and retained data; the operational availability of the ARGUS API and external delivery services lies outside the reported assessment. The associated data-analysis documentation describes the interpretation and selection of monitoring records.[2]

2 Literature and design rationale

Reproducible computational work requires more than a description of an algorithm. Sandve et al. recommend retaining the information needed to reconstruct each result, including the versions of data, software and analysis procedures [1]. Wilson et al. similarly emphasise version control, automated workflows, testing and documentation of interfaces and design decisions [2]. These recommendations motivate the retention of AEGIS inputs and executable preparation steps. The W3C PROV model distinguishes data entities, processing activities and responsible agents, providing a basis for explaining how an output was derived [3].

The Dataflow model distinguishes the time of an event from the time at which a processing stage observes it, and examines the trade-offs between correctness, latency and processing cost [4]. This distinction is relevant to AEGIS even though its collector is currently a periodic script. An assessment can be collected and published long after it was produced. Source timestamps, collection time and publication time are therefore distinguished throughout the workflow. Freshness limits and collection intervals form part of its operational configuration.

Reliability guidance also cautions that retries can amplify load when an upstream service is already failing. Ulrich recommends bounded retries, backoff and deliberate degradation of optional functionality [5]. AEGIS applies request limits and retains the core game when recent monitoring data is unavailable. This separation applies the principle of retaining essential functionality during a dependency failure. The suitability of individual request limits remains subject to operational evaluation.

3 System architecture

The integration comprises a snapshot acquisition path and an incremental monitoring path. Snapshot acquisition supplies the records required for mission construction, alarm interpretation and represen-

*Ilias Akritidis and Georgios Mallousis contributed equally to this work.

[1]Implementation baseline: source revision `b61c190` on branch `readyToSubmit`. Entry points: `tools/fetch_snapshot.py`, `tools/build_game_data.py`, `tools/pi/guardian_live.py`, `.github/workflows/site.yml`, and `web/src/game/live.ts`. File paths in implementation footnotes are relative to the AEGIS repository.

[2]Data-analysis methodology: `doc/data-analysis/README.md`.

tation of the installed sensor networks. The incremental service, hosted on a Raspberry Pi, retrieves recent risk evaluations and alarms. During publication, GitHub Actions combines the application build with a copy of the recent monitoring data to produce a static deployment. The browser accesses these published resources through the application endpoint, **with no direct dependency on the ARGUS API during gameplay**. Figure 1 presents the system context, and Figure 2 identifies the components responsible for acquisition, processing and delivery.

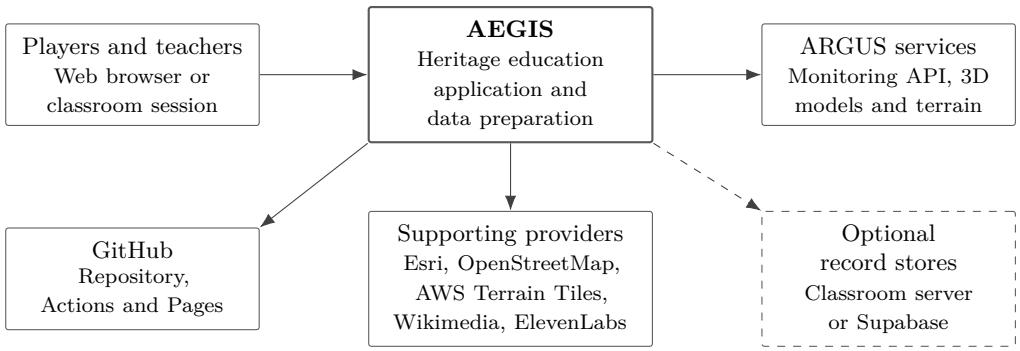


Figure 1: System context. ARGUS supplies monitoring records and spatial assets. AEGIS prepares and publishes these resources for players and teachers. Dashed connections denote optional classroom or shared-storage services.

The browser application uses React, TypeScript, Leaflet, and three.js. A service worker supports the offline core game. A dedicated preparation environment performs snapshot acquisition, retrieves the required spatial assets and generates the application data packs. The Raspberry Pi executes a scheduled Python reader and exposes its output through Caddy and a playit.gg tunnel. GitHub provides source storage, automated checks, and static delivery. Caddy can also serve the built game; its live-data route reads from a separate directory so that publishing a new site does not replace the collector's state[3].

4 Interfaces and acquisition

4.1 ARGUS records

The acquisition scripts target `http://api.argusservices.athenarc.gr/api`. They use paginated JSON responses containing `data` and `meta`, including `last_page` and `total`. The implementation assumes pages contain 15 records in ascending order. Table 1 lists the resources used by AEGIS. All of these acquisition requests use `GET`. RFC 9110 defines `GET` and `HEAD` as safe and idempotent methods, supporting retrieval and the repetition of requests after communication failures [6]. Idempotence concerns the intended server-side effect; it does not guarantee identical responses from a changing dataset.

The snapshot fetcher sends an identifying `User-Agent` and requests JSON. When `ARGUS_API_TOKEN` is present in the environment, it also sends an `Authorization: Bearer` header. The live reader sends an identifying `User-Agent` and accepts JSON, but currently implements no authentication-header mechanism. Authentication requirements are governed by the access policy of the ARGUS deployment; the request mechanisms described here reflect the acquisition clients' configuration.

Table 1: ARGUS resource catalogue. Paths are relative to `/api`; paginated calls append `?page=N`. S: snapshot acquisition; L: incremental reader; R: retained reference snapshot.

Resource	Consumer	Role in AEGIS
`/assets`	S	Identifies the five heritage sites.
`/asset-states`	S	Site metadata and placements of spatial representations.
`/threats`	S	Risk scenarios, categories, ABC scores and priorities.
`/threat-contexts`	S, L	Risk evaluations, probabilities, states and thresholds.
`/events`	S, L	Expert-rule and AI alarms, timestamps, readings and explanatory feature contributions.

[3]Deployment configuration: `tools/pi/guardian-live.service`, `tools/pi/guardian-live.timer`, `tools/pi/Caddyfile`, and `tools/pi/README.md`.

Resource	Consumer	Role in AEGIS
`/policies`	S	Alarm rules and current policy thresholds.
`/installed-instruments`	S	Instrument identifiers, names and location descriptions.
`/installed-devices`	S	Installed-device coordinates.
`/instruments`	S	Links instruments to their types.
`/instrument-types`	S	Instrument-type names used in sensor-card mappings.
`/decisions`	S; R fallback	Decisions recorded after alarms.
`/decision-contexts`	S; R fallback	Relationships between decisions and alarms.
`/monitorings`	R	Sensor placements in the ARGUS three-dimensional viewer.

4.2 Spatial and supporting resources

The spatial fetcher reads the ARGUS terrain descriptors and tiles under `/terrain/<monument>/` and the 3D Tiles descriptors and models under `/tiles/<model>/`. It uses `HEAD` requests to estimate model-tile sizes before downloading them with `GET`. The default model budget is 8 MB per scene. Terrain extraction uses a default 2 m grid, with a maximum acquisition radius of 300 m in the all-scenes mode. These settings bound the spatial subset packaged with the application; they do not describe the full resolution or extent of the upstream datasets. The client checks `robots.txt` before accessing the configured host[4].

Additional services support presentation. Esri World Imagery and OpenStreetMap supply map backgrounds, while AWS Terrain Tiles and Wikimedia Commons support parts of the 3D experience. The OpenStreetMap API supplies outlines used in packaged fallback plans. An offline recording process can generate narration through ElevenLabs; playback uses the resulting audio files. Optional Supabase storage and the local classroom server concern player records, not writes to ARGUS. They remain distinct from the monitoring acquisition path.

5 Processing and data contracts

5.1 Snapshot preparation

`tools/fetch_snapshot.py` retrieves twelve resources, following each response's `meta.last_page`. It pauses 0.25 s between page requests by default and allows four attempts per request, with waits of 2, 4 and 6 s after failures. Each request has a 90 s timeout. Records are deduplicated by identifier and sorted before writing. The output records the resource name, count, completeness flag, collection time, and records. Completeness means that the unique count reaches or exceeds the final reported total. This way the complete count avoids being an atomic transaction across the API. The script refuses to overwrite the original reference snapshot or an existing non-empty destination.

`tools/build_game_data.py` selects the newest available snapshot using its recorded collection metadata and combines it with the educational content used in the application. The retained snapshot dated 12 October 2026 supplies the ten core resource types. Decisions and decision contexts fall back to the original snapshot when absent from the selected input. The spatial scene builder also uses retained monitoring records for viewer placements. Fields drawn from a reference snapshot need not share the collection date shown for the core dataset. Recording that distinction preserves the derivation history needed to interpret a combined output, consistent with the provenance concepts described by PROV-DM [3].

The builder produces `web/public/data/game_data.json` and `docs/data_notes.md`. The former contains the subset needed by the game, including site information, mission risks, sensor mappings, interpretable alarms and their validation rules. The latter records data counts, exclusions and assumptions. Raw asset-state records remain preparation inputs and are not requested by the browser. The detailed rules for alarm filtering and mission construction belong to the accompanying data-analysis

[4]Spatial acquisition and scene preparation: `tools/fetch_argus_3d.py`, `tools/build_preview_3d.py`, and `tools/build_scenes_3d.py`.

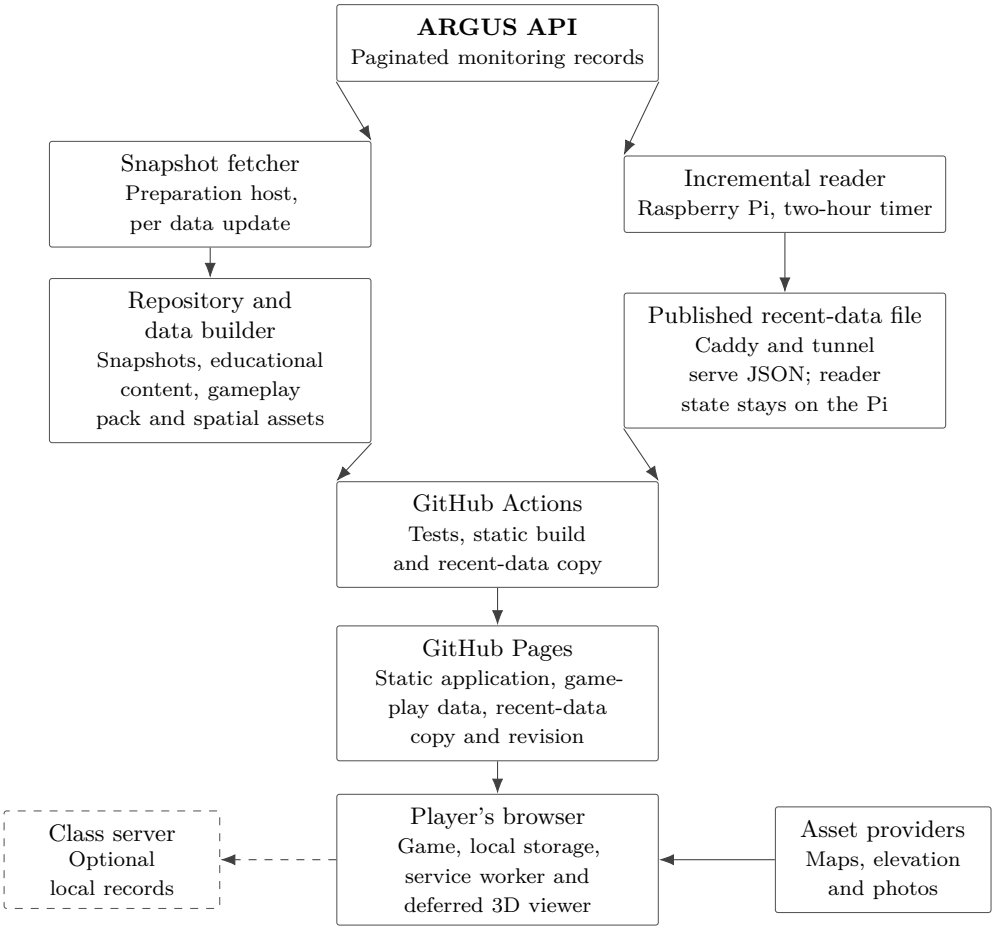


Figure 2: Component and data-delivery view. The snapshot and incremental paths converge during publication. State used by the live reader remains on the Pi, while only its public JSON output enters the website. Optional classroom storage does not communicate with the ARGUS API.

deliverable[5].

5.2 Incremental collection

The live service reads only `threat-contexts` and `events`. For each resource, its cursor stores the last page examined, greatest record identifier, and reported total. A later run revisits the saved page to detect records added to a previously incomplete page, then continues forward. It accepts records with identifiers greater than the saved identifier and processes at most 400 pages per resource per run. Requests are 0.5 s apart; each allows three attempts with waits of 5 and 10 s and a 30 s timeout[6].

If the reported total or page count falls below the saved position, the reader clears the affected resource's state and reads it again from the beginning. This mechanism detects shrinking collections. It does not identify every possible upstream edit, particularly an in-place revision with an unchanged identifier and total.

The reader retains the latest evaluation for each scenario and the ten newest distinct alarms per site. Alarm identity combines its type, rule, timestamp and signal names. When constructing the published risk view, it groups evaluations within twelve hours of the newest evaluation at each site. This heuristic limits the published risk view to evaluations near the newest retained assessment. The configured window establishes temporal proximity between evaluations; membership of a common upstream assessment run remains unverified.

The service writes `state.json` for its next run and `public/argus-now.json` for publication. Each file is written through a temporary file followed by an atomic replacement. If collection fails, the program returns without replacing the previous published file. Atomic replacement applies to each file independently; the state and public output are committed in separate operations.

5.3 Published recent-data contract

The top-level JSON object contains `schema`, `source`, `checked_at`, and `sites`. Schema version 1 identifies the current contract. Each site may contain a risk object and an alarm list. The risk object carries the latest evaluation time and its scenario evaluations. Alarm entries retain their timestamp, type, applicable rule, variables, model score and threshold, and leading feature contribution when available. Fields are reduced to those needed by the browser, and individual copied strings are bounded in length.

Risk timestamps with timezone information are normalised to UTC. Alarm timestamps are retained in ARGUS's timezone-free form; the browser interprets those values as UTC. RFC 3339 identifies unqualified local times as an interoperability problem and specifies timestamps with an explicit relationship to UTC [7]. The interpretation of timezone-free alarm values as UTC is an integration assumption requiring confirmation against the provider's timestamp convention. The file's `checked_at` value describes collection, whereas the nested evaluation and alarm times describe the source events. A recent collection time therefore does not imply that a new assessment occurred at every site. This is an instance of the event-time and processing-time distinction described by Akidau et al. [4].

6 Publication and browser consumption

6.1 Build and static delivery

The publication workflow runs on pushes to `main`, by manual dispatch, and on a nightly schedule at 23:00 UTC. It executes the Python data tests, installs the locked web dependencies, runs the web tests, and builds the application with TypeScript and Vite. A failure prevents the subsequent publication steps[7].

[5] Data preparation and analysis documentation: `tools/build_game_data.py`, `docs/data_notes.md`, and `doc/data-analysis/README.md`.

[6] Incremental-reader parameters and cursor logic: `tools/pi/guardian_live.py`.

[7] Build, validation, fallback and publication steps: `.github/workflows/site.yml`.

After the build, GitHub Actions requests the Pi's published JSON through an address supplied by the repository secret `PI_LIVE_URL`. It accepts a file only if it is valid JSON with `schema=1` and an object-valued `sites` field. This is a structural gate, not full semantic validation of every nested record. If the request or validation fails, the workflow attempts to retain the previous deployed copy. With no valid copy, it publishes the core application without recent monitoring information.

The workflow commits the static output to the `site` branch and writes the source revision to a `VERSION` file. GitHub Pages serves that branch over HTTPS. The Pi's two-hour timer and the website's publication schedule are separate, so browser requests obtain the file copied at the last successful build. Figure 3 summarises the request sequence.

6.2 Selection and freshness

The browser reads `data/game_data.json` and requests `live/argus-now.json` with an 8 s timeout. The live-file loader reuses its in-memory promise for ten minutes, limiting repeated fetches when it is called again. This cache interval is not a guarantee that the interface automatically polls every ten minutes. Nested parsing rejects or omits unusable records before they reach the display[8].

Risk checks remain eligible for fourteen days. Checks no older than 36 hours can receive the "Live from ARGUS" label; older eligible checks receive a less immediate label. Alarm entries remain eligible for sixty days and must pass the same interpretability and plausibility rules supplied by the data builder. An AI alarm, for example, requires a sufficiently dominant feature that maps to a supported sensor explanation. The interface may show an eligible risk view or an interpretable alarm; if neither is available, the recent-data component is omitted while gameplay continues. Values more than one hour in the future are also excluded. The eligibility periods and clock tolerance are application-level display parameters, distinct from the thresholds used by the ARGUS risk models.

6.3 Offline and deferred resources

The service worker stores the core application's build assets and data. It retrieves the data pack from the network first and falls back to its cached copy when the request fails. The recent-data route is network-only and is excluded from service-worker caching. External map tiles are not cached by the application, and the map uses its packaged plans when imagery is unavailable[9].

The three-dimensional viewer loads its scene descriptions, models and terrain on demand. Its code and scene data are excluded from the initial offline pack, and external imagery, elevation tiles or photographs may still require connectivity. Audio clips are cached individually after playback and they are not downloaded in full at installation. Consequently, offline support applies to the core game after a successful first visit, not to every optional resource.

7 Reliability, security and operational limits

Table 2 consolidates the implemented request limits. Bounded retries can recover from some transient failures, while fallback allows the core game to continue without recent monitoring data. The request backoff is linear, whereas the reliability guidance recommends randomised exponential backoff to reduce synchronised retry load [5]. The timer's scheduling jitter does not provide request-level jitter. Request-level retry behaviour therefore remains a candidate for further refinement. Recovery of a large backlog may span several collection runs, depending on the volume accumulated during an interruption.

The acquisition configuration uses HTTP for ARGUS record requests and HTTPS for the public website. The Pi's transfer through the configured tunnel can also use HTTP. The transport protection of the public delivery endpoint consequently does not extend to every upstream transfer. The snapshot client reads an optional bearer token from the environment and does not write it into its snapshot files; its use must follow the access arrangements supplied by ARGUS. Supplying a credential through the environment avoids embedding it in source files but does not protect it during transmission. RFC 6750

[8] Browser parsing and display: `web/src/game/live.ts` and `web/src/components/LiveArgus.tsx`.
[9] Service-worker caching and build exclusions: `web/sw-template.js` and `web/vite.config.ts`.

Table 2: Configured acquisition and delivery limits. Parameters refer to the acquisition and delivery configuration; they do not constitute service-level guarantees.

Component	Spacing or schedule	Timeout	Attempts or bound
Snapshot fetcher	0.25 s between pages	90 s per request	4 attempts; waits 2, 4, 6 s
Incremental reader	0.5 s between pages; 2 h timer	30 s per request	3 attempts; 400 pages per resource per run
Pi service	First run after boot, about 2 min; up to 1 min jitter	15 min per service invocation	Retains prior output after collection failure
Actions live copy	At each successful build	30 s download	Prior valid deployment copy as fallback
Browser live loader	10 min in-memory reuse interval	8 s	No service-worker caching of the live file

requires TLS or equivalent transport security for bearer-token requests [8]. The optional bearer-token path should therefore use a protected transport.

The Pi reader runs as a dedicated system user with its own state directory. The systemd unit specifies a read-only system, protected home directories, private temporary storage and devices, restricted capabilities, and a fifteen-minute runtime limit. Restricting the reader to the permissions needed for its task follows the least-privilege principle articulated by Saltzer and Schroeder [9]. The service configuration specifies the corresponding access restrictions; an independent security assessment remains outside the scope of the verification[10].

All interactions with the ARGUS record API are read-only. Player choices, scores and observations do not enter that API. In the default device-storage mode, these records remain in browser storage. An explicitly configured classroom server or shared store introduces a separate destination for supported player records. Observation reports leave the device through an explicit user action, such as saving, printing or preparing an email. These optional record-sharing functions operate independently of the monitoring feed and preserve the separation between heritage-data acquisition and player-generated information.

8 Reproducibility and verification

8.1 Local reproduction

The data tools require Python 3.9 or later and use the standard library. The web application requires Node.js 20 or later and installs its dependencies from `web/package-lock.json`. The committed snapshots and packaged data allow local inspection without contacting ARGUS. Retaining the original inputs, software revision and executable preparation commands supports the reconstruction of computational results advocated by Sandve et al. [1]. The following commands run from the repository root and validate the existing pack before building and serving the application:

```
python3 -I tools/build_game_data.py --check
python3 -I -m unittest discover -s tools/tests -t tools
cd web
npm ci
npm test
npm run build
npm run preview
```

Omitting `-check` in the first command regenerates the pack from the selected retained snapshot. To acquire a fresh snapshot, run the following with the access configuration supplied by the organiser; the destination must be new or empty:

```
python3 -I tools/fetch_snapshot.py --out snapshot_review
python3 -I tools/build_game_data.py --snapshot snapshot_review
python3 -I tools/build_preview_3d.py
python3 -I tools/build_scenes_3d.py
```

[10] Deployment configuration: `tools/pi/guardian-live.service`, `tools/pi/guardian-live.timer`, `tools/pi/Caddyfile`, and `tools/pi/README.md`.

```
python3 -I tools/fetch_argus_3d.py all
```

The new snapshot command retrieves twelve resources; it does not recreate the original collection of all ARGUS resources. Spatial generation still requires the retained monitoring and placement inputs. Network-backed commands also depend on the continued availability and permitted use of upstream services. Fallback map plans and narration have their own preparation tools, `tools/fetch_backdrops.py` and `npm run narration`, respectively.

A single incremental collection can run on any suitable computer, independently of systemd:

```
python3 -I tools/pi/guardian_live.py --out /tmp/aegis-live-review
```

This creates `/tmp/aegis-live-review/state.json` and `/tmp/aegis-live-review/public/argus-now.json`; it does not publish them automatically. Deployment of the Pi service uses the supplied systemd unit, timer and Caddy configuration. Publication to GitHub Pages uses the repository workflow and its configured live-file address. The optional classroom server can serve a built game locally with `python3 -I tools/class_server.py`; LAN access requires its explicit `-lan` option.

9 Conclusion

The workflow supports a reproducible educational application by separating the preparation of gameplay data from the collection of recent monitoring records. A static publication step gives the browser a compact interface to ARGUS-derived information, while retained inputs and fallback behaviour preserve the core experience when upstream services are unavailable. Its principal operational constraints are the assumptions of the incremental reader, the delay introduced by publication, the HTTP acquisition paths, and the connectivity needed by optional spatial resources. These constraints define the conditions under which the delivered workflow should be reproduced and interpreted.

References

[1] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, "Ten Simple Rules for Reproducible Computational Research," *PLOS Computational Biology*, vol. 9, no. 10, e1003285, 2013. https://doi.org/10.1371/journal.pcbi.1003285.

[2] G. Wilson et al., "Best Practices for Scientific Computing," *PLOS Biology*, vol. 12, no. 1, e1001745, 2014. https://doi.org/10.1371/journal.pbio.1001745.

[3] L. Moreau and P. Missier, eds., *PROV-DM: The PROV Data Model*. W3C Recommendation, 30 April 2013. https://www.w3.org/TR/2013/REC-prov-dm-20130430/.

[4] T. Akidau et al., "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing," *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1792–1803, 2015. https://doi.org/10.14778/2824032.2824076.

[5] M. Ulrich, "Addressing Cascading Failures," in *Site Reliability Engineering: How Google Runs Production Systems*, B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, eds. O'Reilly Media, 2016, ch. 22. https://sre.google/sre-book/addressing-cascading-failures/.

[6] R. Fielding, M. Nottingham, and J. Reschke, eds., *HTTP Semantics*. RFC 9110, June 2022, secs. 9.2.1–9.2.2. https://doi.org/10.17487/RFC9110.

[7] G. Klyne and C. Newman, *Date and Time on the Internet: Timestamps*. RFC 3339, July 2002, secs. 4–5. https://doi.org/10.17487/RFC3339.

[8] M. Jones and D. Hardt, *The OAuth 2.0 Authorization Framework: Bearer Token Usage*. RFC 6750, October 2012, sec. 5.3. https://doi.org/10.17487/RFC6750.

[9] J. H. Saltzer and M. D. Schroeder, "The Protection of Information in Computer Systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975. https://doi.org/10.1109/PROC.1975.9939.

A Workflow interfaces and inspection links

The published application and its data interfaces are listed below for inspection. The addresses identify the configured deployment. Availability and revision correspondence at these endpoints require deployment-level verification.

Application: `https://aloualou56.github.io/argus_trackA/`

Gameplay dataset: `https://aloualou56.github.io/argus_trackA/data/game_data.json`

Recent monitoring data: `https://aloualou56.github.io/argus_trackA/live/argus-now.json`

Source repository: `https://github.com/aloualou56/argus_trackA`

The original detailed architecture sources remain in `doc/api-workflow/`: `1-context.mmd`, `2-containers.mmd`, and `3-sequence.mmd`, with corresponding SVG and PNG files. The figures present the system context, component responsibilities and request sequence, with interface details and operational parameters specified in the corresponding tables.

Table 3: Principal file contracts and responsibility boundaries.

Artifact	Producer	Contents and consumer
Snapshot resource JSON	Snapshot fetcher	`resource`, `count`, `complete`, `fetched_at`, `records`; read during preparation.
`game_data.json`	Data builder	Validated gameplay subset, provenance and alarm-selection rules; read by the application.
`state.json`	Pi reader	Cursor positions, retained risks and alarms; read by subsequent collection runs. Not part of the public feed.
`argus-now.json`	Pi reader	Schema version, collection time and per-site risks/alarms; copied by Actions, parsed by the browser.
Scene files, models, terrain	Spatial preparation	Site geometry and sensor placements; loaded when the user opens the 3D view.
`VERSION`	GitHub Actions	Source commit served by the published website; supports revision checks.

B Acquisition and delivery sequence

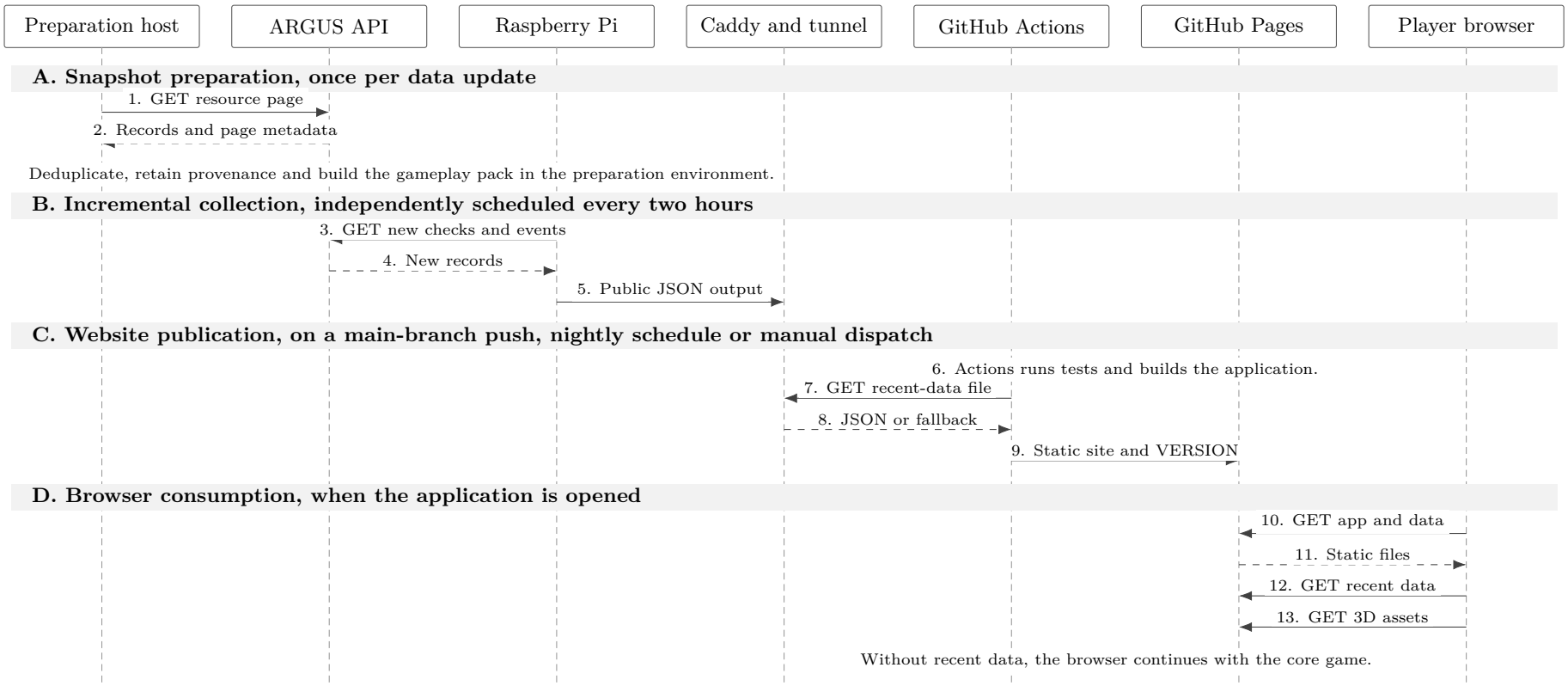


Figure 3: Representative sequence of snapshot acquisition, incremental collection, static publication and browser consumption. Horizontal bands separate independently scheduled processes; they do not imply that a snapshot is refreshed at every live-reader invocation. Return arrows show representative responses, and the text specifies retries, validation and fallback behaviour.